18. Project Layout: Libraries vs Services

18. Project Layout: Libraries vs Services

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1 day
Slug: `18-project-layout`

Learning objectives

- Choose layouts for libraries vs services
- Use cmd/, internal/, pkg/ judiciously
- Keep main thin — logic in packages
- Avoid premature micro-packages

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Layout signals intent: library vs service, public API vs internal. Good structure reduces import cycles and onboarding time.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `cmd/` for mains, `internal/` for private packages, root for public libs.
- Avoid `pkg/` unless your org standardizes on it.
- Keep modules cohesive; split when versioning demands.
- One module per repo is common; multi-module is advanced.
- Configs and migrations live near the service that owns them.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-project && cd /tmp/gw-project
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

// Example layout commentary printed as a checklist.
func main() {
    layout := []string{"cmd/api", "internal/auth", "internal/store", "go.mod"}
    for _, p := range layout {
        fmt.Println(p)
    }
}
```

Run it:

```
go run .
```

Expected output:

```
cmd/api
internal/auth
internal/store
go.mod
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Sketch folders for a tiny HTTP API.
2. Move a helper under internal/ and import it from cmd.
3. Try importing internal from a sibling module — observe failure.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Common service layout

```
cmd/api/main.go      # wiring only
internal/server/     # private app code
internal/store/
go.mod
```

Libraries

Export a stable API from the module root or clear subpackages. Semantic import versioning: `/v2` path for major version 2+.

Guidelines

- `internal/` for code you do not want strangers to import
- Prefer fewer packages until boundaries hurt
- Config via env/flags at main; pure functions underneath for tests

Failure modes to watch while coding

- Copying a giant layout template for a 200-line tool
- Business logic inside main.go
- Exporting everything from pkg/ without need

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- src-layout vs flat — Go favors module-root clarity.
- Django apps ≈ packages with clear boundaries.
- **init**.py noise does not exist; directories are packages with Go files.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Start simple. Premature micro-packages create cycles. Split on domain seams you can name.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Project Layout: Libraries vs Services" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Copying a giant layout template for a 200-line tool
- Business logic inside main.go
- Exporting everything from pkg/ without need

Exercises

Work in `exercises/18-project-layout/`. Read that folder's `README.md` before you code.

What you will build

Answer layout quiz stubs / implement the tiny helper the exercise expects.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/18-project-layout/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What belongs in cmd/?
2. Why internal/?
3. When introduce a new package?

Further reading

- https://go.dev/doc/modules/layout
- https://github.com/golang-standards/project-layout